

React

styled component

```
const ReactButton = (props) => {  
  const style = {  
    color: 'white',  
    backgroundColor: 'purple'  
  }  
  
  return <button style={style}>{props.children}</button>  
}
```

まず、styled componentを使わなかったコードである。

これを見ると、<button>tagを最上位にする。

```
import styled from 'styled-components';  
const StyledButton = styled.button`  
  color: white;  
  background-color: purple;  
`;  
;
```

これはstyled componentを使ったコードである。

見た通りstyled componentを使ったらCSSをそのままで入力することができる。

```
t> npm install styled-components
```

```
import styled from 'styled-components';

const SimpleButton = styled.button``;

function App() {
  return (
    <div className="App">
      <SimpleButton>Simple</SimpleButton>
    </div>
  );
}
```

styled-componentsをインストールして、このようにする。

そして``のなかにcssを入力する。

Simple

```
const SimpleButton = styled.button`
  color: white;
  background-color: green;
`;
```

Simple

このようにcssを設定する。

Context API

```
.root,
.root div {
  border: 10px solid gray;
  margin: 10px;
  padding: 10px;
}
```

```
export default function App() {  
  return (  
    <div className="root">  
      <h1>Hello World!</h1>  
    </div>  
  );  
}
```

まず、rootクラスにcssを設する。

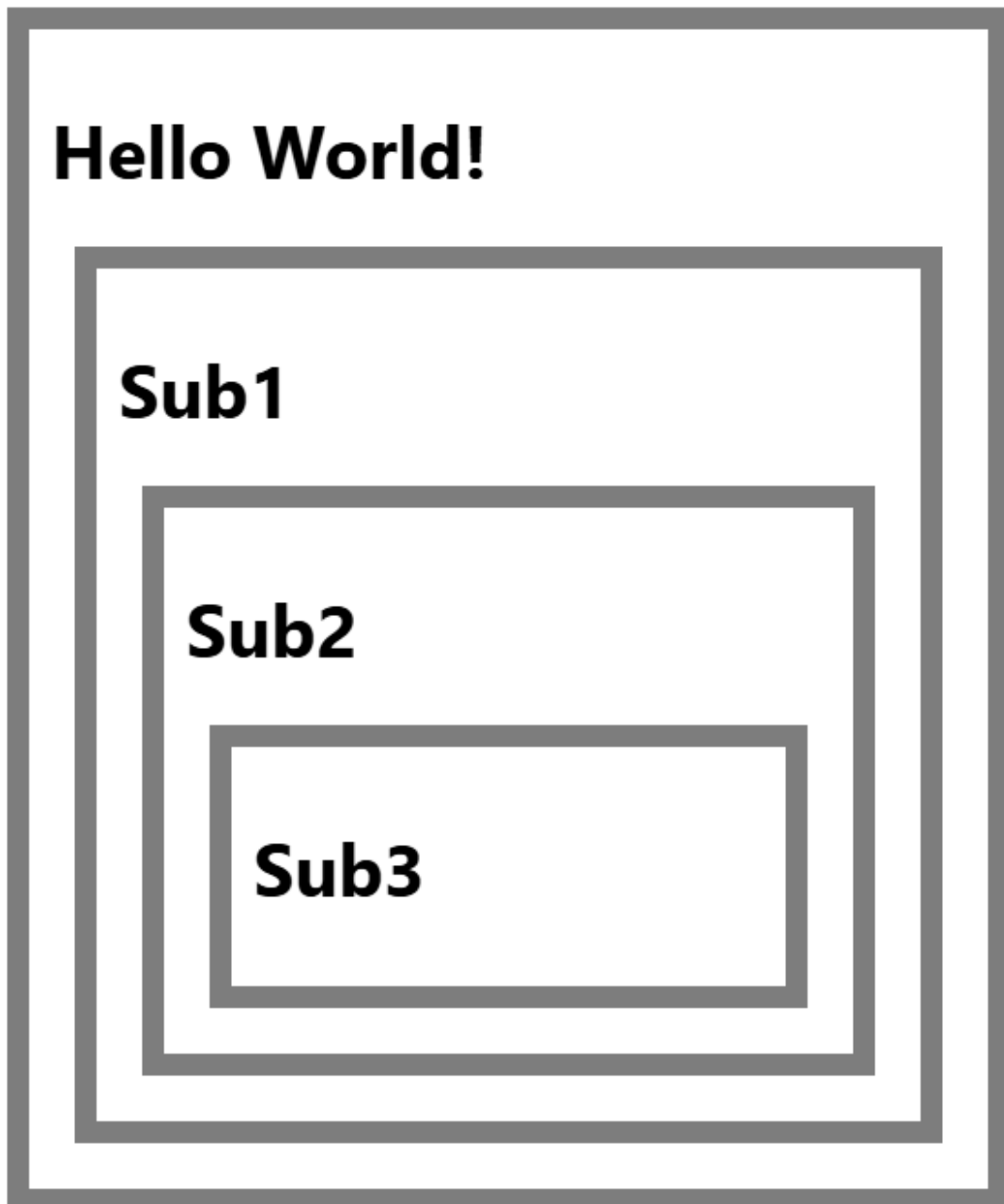
```
import React from 'react';
import './style.css';

export default function App() {
  return (
    <div className="root">
      <h1>Hello World!</h1>
      <Sub1 />
    </div>
  );
}

function Sub1() {
  return (
    <div>
      <h1>Sub1</h1>
      <Sub2 />
    </div>
  );
}

function Sub2() {
  return (
    <div>
      <h1>Sub2</h1>
      <Sub3 />
    </div>
  );
}

function Sub3() {
  return (
    <div>
      <h1>Sub3</h1>
    </div>
  );
}
```



このようなコードを入力して、Context APIを使う準備をする。

```
const themeDefault = {border: '10px solid green'};  
const themeContext = createContext(themeDefault);
```

reactにcontextを使用すると、createContext関数をimportする。

```
import React, { createContext, useContext } from 'react';
```

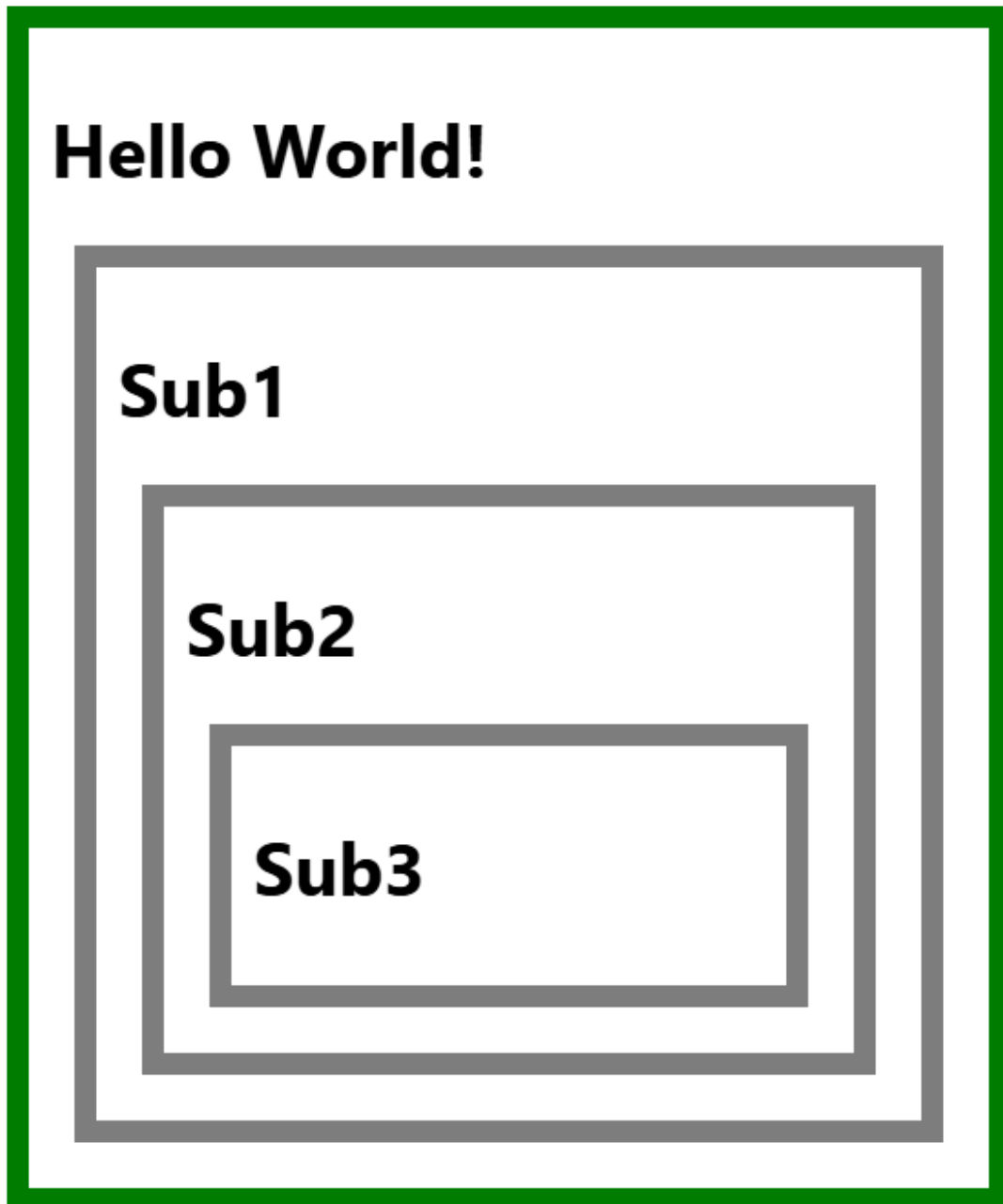
```
const theme = useContext(themeContext);
console.log('theme', theme)
```

使用方法はuseContextをimportしてuserContext関数をcomponentで使用すればよい。

```
theme ▶ {border: "10px solid green"}
theme ▶ {border: "10px solid green"}
> |
```

consoleをみると、contextを生成する時にcreateContextに伝えたthemeDefault値がuseContextのreturnになることが見える。

```
return (
  <div className="root" style={theme}>
    <h1>Hello World!</h1>
    <Sub1 />
  </div>
);
```



そして、このようにcomponentのstyleを変えることができる。

```

const theme = useContext(themeContext);
return (
  <div className="root" style={theme}>
    <h1>Hello World!</h1>
    <Sub1 />
  </div>
);

```

```

function Sub1() {
  const theme = useContext(themeContext);

  return (
    <div style={theme}>
      <h1>Sub1</h1>
      <Sub2 />
    </div>
  );
}

```

```

function Sub2() {
  const theme = useContext(themeContext);

  return (
    <div style={theme}>
      <h1>Sub2</h1>
      <Sub3 />
    </div>
  );
}

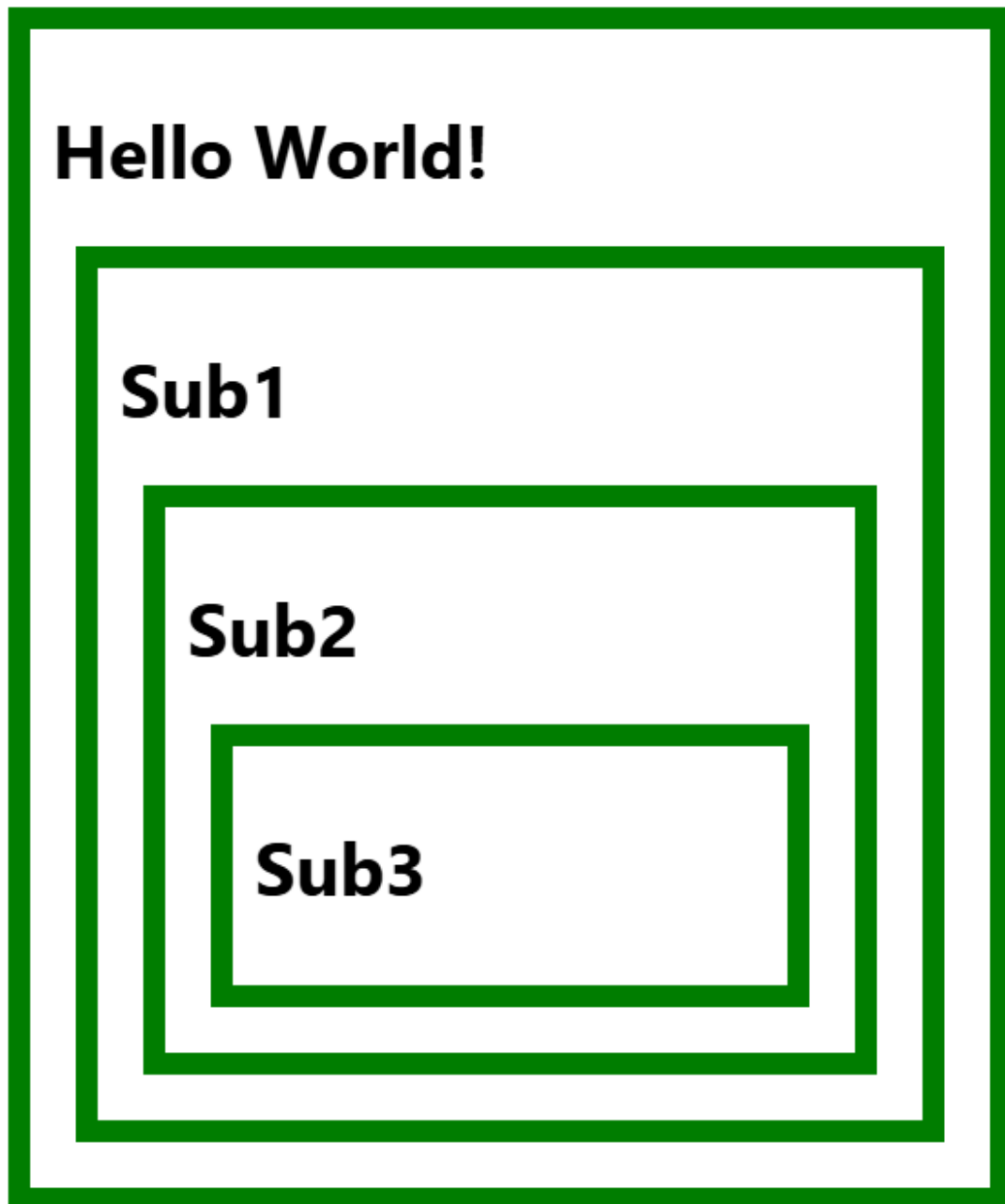
```

```

function Sub3() {
  const theme = useContext(themeContext);

  return (
    <div style={theme}>

```

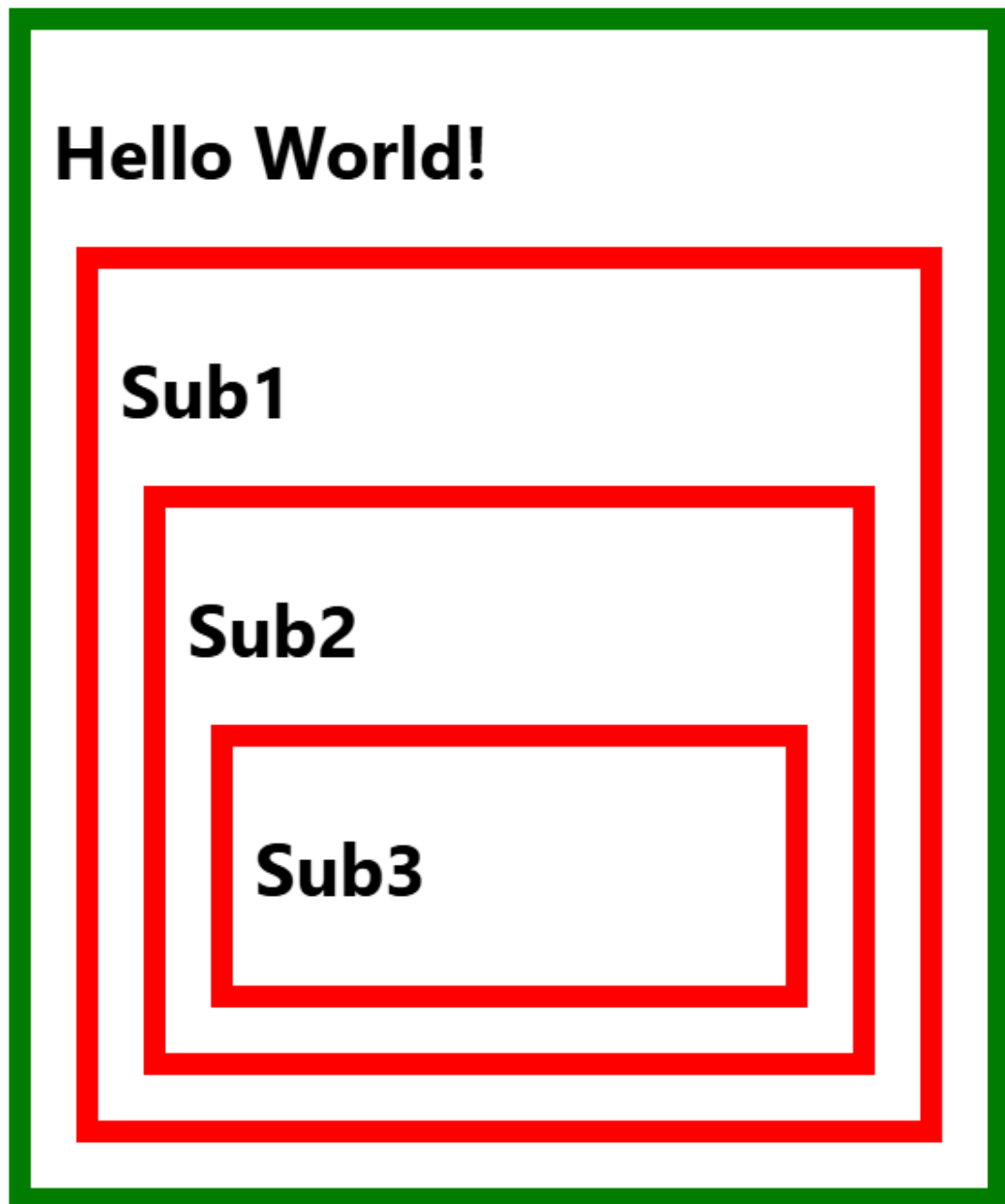


もちろんすべてのcomponentにコードを入力して全部変えることもできる。
しかし、大量のcomponentを一つ一つ入力することは難しい。

そのためにProviderを使う。

```
export default function App() {  
  const theme = useContext(themeContext);  
  return (  
    <themeContext.Provider value={{ border: '10px solid red'}}>  
      <div className="root" style={theme}>  
        <h1>Hello World!</h1>  
        <Sub1 />  
      </div>  
    </themeContext.Provider>  
  );  
}
```

Providerを使って上位のcomponentをラップすれば



すべての下位componentはかえる。